AI Edge (Computing)

6. Deploying a model with TFLM

hogeschool vives

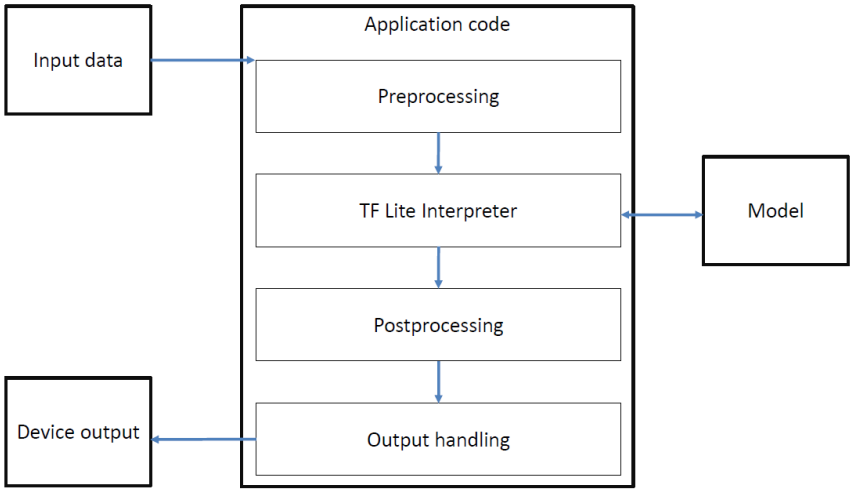
1

Tensorflow Lite for Microcontrollers

- **Open-source C++ library & interpreter for TFLite models**
- Requires:
  - 32-bit platform
  - C++ 17
  - Model in C byte array (model.h-file)
  - At least 16 KB memory
- Available at: https://github.com/tensorflow/tflite-micro

Tensorflow Lite for Microcontrollers

Application Architecture



3

Tensorflow Lite for Microcontrollers

Importing libraries

This is the bare minimum, other lib's available (error reporting & performance)

```
#include "mbed.h"

#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"

#include "model.h"

using namespace tflite;
```

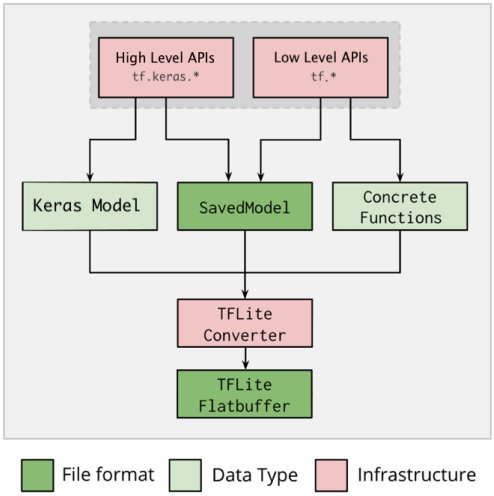

4

Tensorflow Lite for Microcontrollers

Flatbuffer

- Our model header file

- **Serialized data**
- **Memory efficient & fast**
- **Flexible**
- **Strongly typed**
- **Cross platform**



5

Tensorflow Lite for Microcontrollers

Object setup

Objects for further reference

```
//Map the model into a usable data structure. This doesn't involve any
//copying or parsing, it's a very lightweight operation.
const Model* model = GetModel(g_model);

// Interpreter, input and output structures
MicroInterpreter* interpreter = nullptr;
TfLiteTensor* input = nullptr;
TfLiteTensor* output = nullptr;
```


6

Tensorflow Lite for Microcontrollers

Memory setup

Create an area of memory to use for input, output, and intermediate arrays.
Minimum arena size, at the time of writing. This is a "guess".

```
// Allocate memory for the tensors
const int ModelArenaSize = 2468; //2468;
// Extra headroom for model + alignment + future interpreter changes
const int ExtraArenaSize = 560 + 16 + 100;
const int TensorArenaSize = ModelArenaSize + ExtraArenaSize;
uint8_t tensor_arena[TensorArenaSize];
```


Tensorflow Lite for Microcontrollers

Memory setup

```
// Extra project variables
const float Pi = 3.14159265359f;
const float xrange = 2.f * Pi;
const int InferencesPerCycle = 100;
int inference_count = 0;

// Led to indicate the resulted values
PwmOut nucleo_led(LED1);
```

Tensorflow Lite for Microcontrollers

Operations resolver

Resolver for the operations

1. **All operations**
2. **Some operations (only those used in your model)**

```
// static AllOpsResolver resolver;
// OR
static MicroMutableOpResolver<1> resolver;
resolver.AddFullyConnected();
```


hogeschool vives

Tensorflow Lite for Microcontrollers

Interpreter

Interpreter for the model that uses all the previously declared classes and variables

```
// Build an interpreter to run the model with.
static MicroInterpreter static_interpreter(model,
  resolver, tensor_arena, TensorArenaSize, NULL);
interpreter = &static_interpreter;

// Allocate memory from the tensor_arena for the model's tensors.
interpreter->AllocateTensors();
```


hogeschool vives

Tensorflow Lite for Microcontrollers

Pointers for the inputs and outputs

Interpreter for the model that uses all the previously declared classes and variables

```
input = interpreter->input(0);
output = interpreter->output(0);
```


11

Tensorflow Lite for Microcontrollers

Creating an input value

Generate the next value

```
float generateNextXValue() {
    float position = (float) inference_count / InferencesPerCycle;
    float x_value =  position * xrange;
    inference_count = (inference_count + 1) % InferencesPerCycle;
    return x_value;
}
```


12

Tensorflow Lite for Microcontrollers

Creating an input value

Sidenote: when the input is more complex (multi dim)

```
o [1,2,3]
input->data.f[0] = 1;
input->data.f[1] = 2;
input->data.f[2] = 3;

o [[1,2,3],[4,5,6]]
input->data.f[0] = 1;
input->data.f[1] = 2;
input->data.f[2] = 3;
input->data.f[3] = 4;
input->data.f[4] = 5;
input->data.f[5] = 6;
```


13

Tensorflow Lite for Microcontrollers

Inference

Inference (with timing measurements)

```
float inference(float x) {
  Timer t;
  input->data.f[0] = x;
  t.start();
  interpreter->Invoke();
  t.stop();
  auto us = t.elapsed_time().count();
  printf("Inference time: %llu us\n", us);
  return output->data.f[0];
}
```


14

Tensorflow Lite for Microcontrollers

Output processing

Do something with the output value (change the LED brightness and print the value)

```
void setLed(float value) {
  nucleo_led = (value + 1) / 2;
  // sensortile_led = value < 0.0f ? 0 : 1;
}

void printValues(float x, float y) {
  printf("x_value: % 2.3f, y_value: % 02.3f\n", x, y);
}
```


15

Tensorflow Lite for Microcontrollers

Extra requirements

In Mbed this is run indefinetly using an EventQueue and calling this every 10 ms.

```
// Global declaration:
EventQueue queue;

// In the main file:
queue.call_every(10ms, run_once);
queue.dispatch_forever();

// Function
void run_once() {
    float x_value = generateNextXValue();
    float y_value = inference(x_value);
    setLed(y_value);
    printValues(x_value, y_value);
}
```


16

Tensorflow Lite for Microcontrollers

Next step… Project time!

Now run this!

Compare the code with the required code when using the Edge Impulse C++ library

- **Similarities?**
- **Differences?**

Project 2:

- **Train and deploy an AI application model on an embedded device wo Edge Impulse**